



University of Antwerp
| Faculty of Arts

An Introduction to Command Line

DTA Bootcamp 2025 – 2026

Tutor: Jens Van Nooten

Assistant: Victor De Marez

Goals for today's session

1. Learn what command line is
2. Learn our first commands
3. Write and run our first python script in command line



System Check

- Did all Windows users install git bash (<https://git-scm.com/downloads/win>)?
- Did all Windows users activate WSL (<https://winsides.com/how-to-enable-windows-subsystem-for-linux-in-windows/>) and install Ubuntu (<https://apps.microsoft.com/detail/9pdxgncfsczv>)?

Instructions for installing WSL and Ubuntu (WINDOWS!)

- In Powershell/terminal: `wsl --install`
- Go to <https://ubuntu.com/desktop/wsl> and follow the link to the Windows Store
- Open Ubuntu and wait for the installation to finish
- Enter (new) UNIX username and password
- All files created in Ubuntu will be stored under a separate directory
 - To access already existing files on your pc, type `cd /mnt/c`

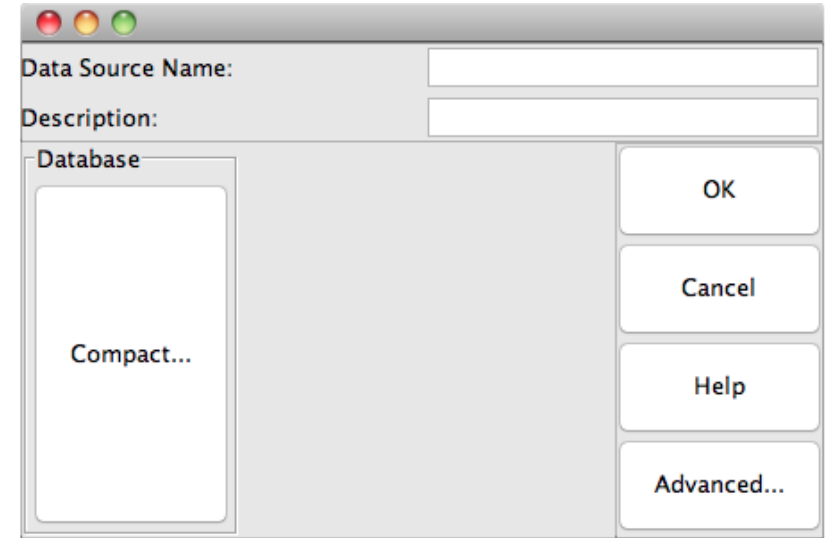
Online terminals

- **In case you are unable to install Ubuntu as a Windows user:**
 - <https://onlinelinuxterminal.com/terminal/>
- **Limited, but enough capabilities to follow along during this class**

Command Line Interface, Terminals and Shells

Command Line Interfaces

- **Command Line Interface (CLI):** Allows you to 'talk' and interact with your computer by entering commands
 - ⇔ Graphical User Interface (GUI)
- **Allows you to do a lot of things more efficiently:**
 - Navigating directories
 - Removing, renaming, creating files
 - Running programs/scripts
 - ...



```
jensvn@JenslaptopHP: ~  
jensvn@JenslaptopHP:~$ echo 'Hello, world! :)'  
Hello, world! :)  
jensvn@JenslaptopHP:~$
```

Terminals and Shells

- **Terminal:** tool that allows you to run commands
 - MAC terminal, Windows terminal...
- **Shell:** the interpreter of the commands you type
 - **Different shell, different commands**
 - MAC/Linux: bash
 - Windows: Powershell
 - That's why Windows users installed Ubuntu, which uses the bash shell!



Why use CLI?

Why use it?

- **Efficiency:** Some tasks are faster to perform using commands rather than clicking through menus
 - Quickly rename incorrectly formatted filenames in a directory
- **Flexibility:** You can combine commands to perform complex tasks, automate processes, and access hidden system functionality
 - Schedule directory clean-ups, automatically zip files...
- **Power:** Many powerful tools for programming and data analysis work through the command line

→ All very useful when working on remote servers, which you will encounter during your internship, thesis work, or at a job!

Let's try some commands!

Some basic commands: pwd

- **pwd**: shows the current working directory (where you are in your system)
 - First **/**: root, or the top most directory in your system

```
jensvn@JenslaptopHP:~$ pwd  
/home/jensvn
```

Keep in mind: commands are case sensitive! Pwd != pwd != PWD

Some basic commands: ls

- **ls**: Shows a list of files and directories

- Multiple 'flags' (= command options):

- **-l**: shows permissions, number of links/directories in this dir, user, group, size in bytes, date of last modification and name
 - **-a** to include hidden files
 - **-s** to include file size
 - **-R** to recursively list directory tree

```
jensvn@JenslaptopHP:~$ ls -l
total 12
-rw-r--r-- 1 jensvn jensvn  40 Feb 13  2024 test1.txt
-rw-r--r-- 1 jensvn jensvn  21 Feb 13  2024 test2.txt
drwxr-xr-x 2 jensvn jensvn 4096 Sep 29 15:02 test_dir
```

- **Also works with**

- Specific directories: **ls test_dir/**
 - Wildcards: **ls *.txt**

Some basic commands: cd

- **cd**: go down or up in your filesystem
 - First use **ls** to see which directories you can move to
 - Then, do **cd [DIRECTORY]**
 - cd **test_directory**
 - Notice how the path changed from ~ to ~/test_dir !

```
jensvn@JenslaptopHP:~$ ls
test1.txt  test2.txt  test_dir
jensvn@JenslaptopHP:~$ cd test_dir/
jensvn@JenslaptopHP:~/test_dir$
```

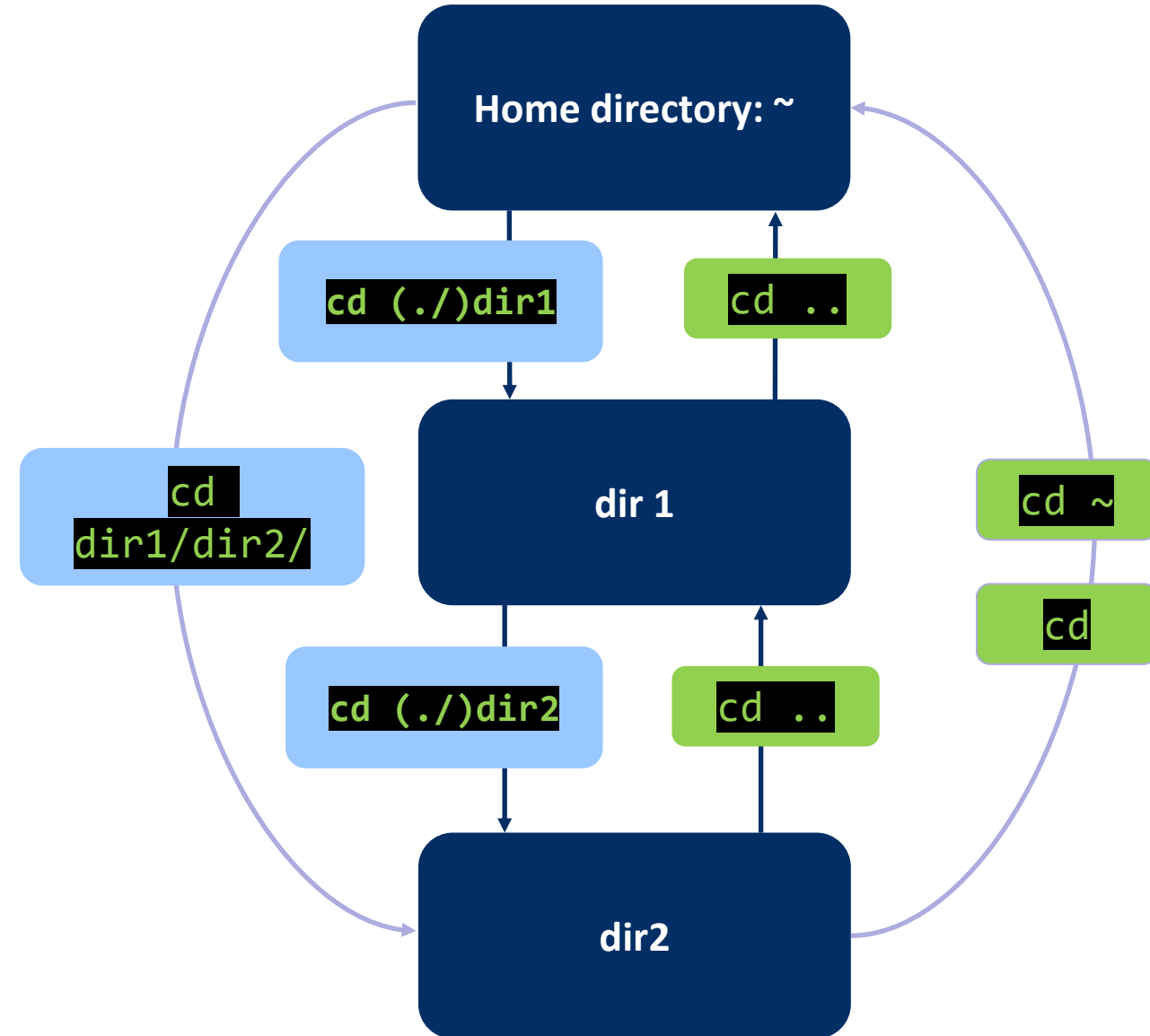
Some symbols to know

■ Relative paths:

-  : current location
-  : parent directory
-  : home directory in a path (user-specific; not the root!!!)
 - ~/path/to/directory

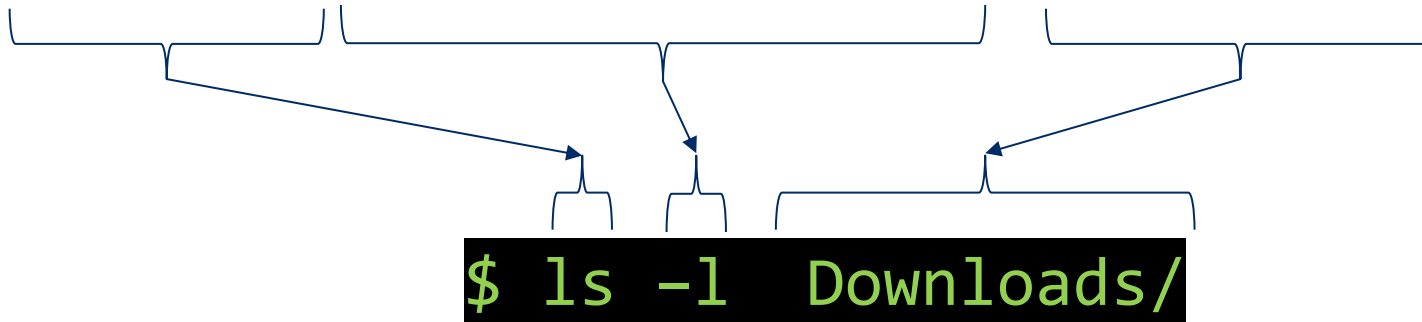
Some symbols to know

- So, **moving back up**: `cd ..`
- **Moving down**: `cd DIR`
- Moving to your home directory: `cd ~` or `cd`
- You don't always need to specify the full path from the root up:
`cd /home/User/dir1/dir2`
=
`cd ../dir1/dir2`
 - You can even leave out `../` in this case



Command structure

- Basic structure of most commands you will use:
command [- flag/option(s) [value]] [arguments]



- `$` = beginning of command, don't type this 😊

- A command can have zero or more options**

An option can have zero or one value

- A command can have zero or more arguments**

- Arguments always come after options**

Creating files and directories: mkdir, touch and nano

- **Let's make some directories and files!**

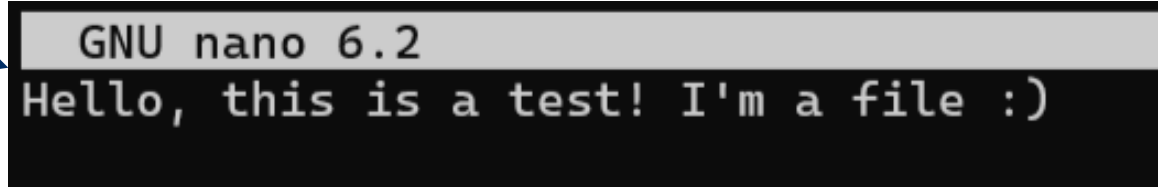
- **mkdir**: allows you to create an empty directory (in working directory)
 - `mkdir test_dir`
- **touch**: creates an empty file (in working directory)
 - `touch test_file.txt`
 - You can create files in other directories than your pwd: `touch test_dir/test_file.txt`
- **nano**: creates an empty file if file doesn't exist already
 - Otherwise it opens the existing file
 - `nano test_file.txt`
 - To exit: CTRL + X and press "Y" to save changes (press "N" to discard changes)

Displaying the contents of files: nano, cat, head, tail

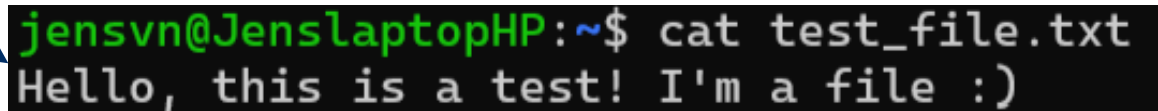


- Now that we've created some files, we want to view them

- **nano**: opens file in editor
- **cat**: print out content
- **head**: show n first lines of file
 - head test_file.txt
 - head -n 5 test_file.txt
- **tail**: show n last lines of file
 - tail test_file.txt
 - tail -n 5 test_file.txt



GNU nano 6.2
Hello, this is a test! I'm a file :)

A terminal window showing the GNU nano 6.2 text editor. The first line is the prompt 'GNU nano 6.2' and the second line is the file content 'Hello, this is a test! I'm a file :)'. Two blue arrows point from the 'nano' and 'head' entries in the list to this terminal window.

jensvn@JenslaptopHP:~\$ cat test_file.txt
Hello, this is a test! I'm a file :)

A terminal window showing the command 'cat test_file.txt' being executed. The output is 'Hello, this is a test! I'm a file :)'. A blue arrow points from the 'cat' entry in the list to this terminal window.

Copying and moving/renaming: cp and mv

- **Backing up your files is always a good idea:**

- Copying file(s)/dir(s): `cp`
- Comes with multiple options, including `-r` (recursive)
- `cp [FILE_NAME] [COPIED_FILE_NAME]`
 - `cp test_file.txt test_file_copy.txt`
 - `cp test_file.txt test_dir/test_file_copy.txt`
 - `cp -r test_dir test_dir_copy`

- **Moving files: `mv`**

- Also used to rename files
- Similar syntax to `cp`
- Recursive: `-r`
- `mv [FILE] [DESTINATION]`
 - `mv test_file.txt test_dir/test_file.txt`

Removing files: rm

- **BE VERY CAREFUL WITH THE FOLLOWING COMMANDS! REMOVING FILES CANNOT BE UNDONE!**
 - Always doublecheck **where** you are in your system
 - Doublecheck **what** you want to remove
 - *Make back-ups or use trash (or other variants)*
- **Removing files can be done with `rm`**
 - `rm test_file.txt`
 - **!!! Recursive: -r**
 - Removes directory and all files in it: `rm -r test_dir`
- **Removing an empty directory: `rmdir`**
 - `rmdir test_dir`

Running a python script

- **Let's create a python script!**

- Open an empty file
- Write a very basic script
- Save and close the file
- Run the file

(`touch`, `nano...`)

(print something)

(ctrl/cmd + x, then press 'y')

(`python test_script.py`)

- **Of course, you can use code editors (VScode, Sublime Text, etc.) to create Python files more easily (not using CLI)**

Excursion: Redirecting and Appending

- You can also create files by redirecting commands (>):

- `ls test_dir > output_ls.txt`
- `echo 'hello, world!' > hello.txt`

- Appending is quite easy (>>):

- `echo 'goodbye, world!' >> hello.txt`

- Especially useful for logging python scripts!

- `python test_script.py > output.txt`

In summary

- Moving around:
- Creating files and dirs:
- Opening files:
- Copying files and dirs:
- Moving files and dirs:
- Running a python script:

```
cd
```

```
mkdir, touch, nano
```

```
nano, vi(m)
```

```
(s)cp
```

```
mv
```

```
python / python3
```

Let's go deeper!

Other use cases for CLI

- Version control of code:
- Downloading packages:
- Downloading python packages:
- Anaconda-related commands:
- Transfer data:
- Testing your network connection:
- Connecting to a remote server:

git

brew / winget

pip / uv

conda

curl

ping

ssh

Other useful commands

- **Look up these commands:**

- `grep` : lookup in files
 - `grep "STRING" [FILE]`
- `wc` : get word counts
 - `wc [FILE]`
- `less` : open file in new window
 - `Less [FILE]`

- **Sequentially executing commands: pipelines**

- Multiple commands separated by `|`
 - `command_1 | command_2`
 - `ls | grep 'test'`

Bonus: Path

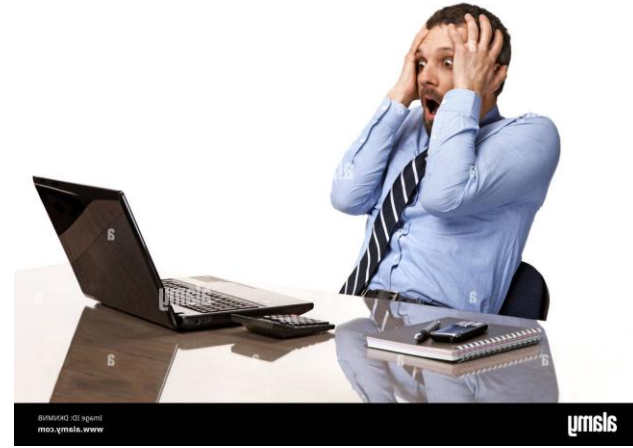
- **How does your terminal know which commands it can run?**
 - Path! (usually located in `/usr/bin/` or `/usr/local/bin/`)
 - Directory that keeps track of all the commands you can run
- **If you try to run a command in a terminal:**
 - It checks the path
 - If the corresponding directory for the command is found, the command is executed

Bonus: Path

```
jensvn@JenslaptopHP:~$ cd /usr/bin/
jensvn@JenslaptopHP:/usr/bin$ ls
NF                cpp-11            gcov-11           lsattr            piconv            sha224sum         uname
['               crontab          gcov-dump         lsb_release       pidof             sha256sum         unattended-upgrade
aa-enabled        csplit           gcov-dump-11     lsblk            pidwait          sha384sum         unattended-upgrades
aa-exec          ctail           gcov-tool        lscpu            pinentry         sha512sum         uncompress
aa-features-abi  ctstat          gcov-tool-11     lshw            pinentry-curses  shasum           unexpand
add-apt-repository curl            gdbus            lsipc           ping            showconsolefont  unicode_start
addpart          cut            gencat          lslocks         ping4           showkey          unicode_stop
addr2line        cvtsudoers     geqn            lslogins        ping6           shred           uniq
apport-bug       dash           getconf          lsmem          pinky           shuf            unlink
apport-cli       date           getent          lsmod          pip            size           unlnma
apport-collect   dbus-cleanup-sockets dbus-daemon      lsof           pip3           skill          unshare
apport-unpack    dbus-launch    dbus-monitor     lspci          pip3.10        slabtop         unsquashfs
apropos          dbus-run-session dbus-send        lspgpot        pkaction       sleep          unxz
apt              dbus-update-activation-environment dd              lzcat          pkcheck        slogin         update-alternatives
apt-add-repository dbus-send      git             lzcmp          pkcon          snap            update-mime-database
apt-cache        dbus-update-activation-environment git-receive-pack lzdiff         pkexec         snapctl        uptime
apt-cdrom        dbus-uuidgen  git-shell       lzfgrep        pkill          snapfuse       usb-devices
apt-config       dealloctv     git-upload-archive lzgrep         pkmon          snice          usbhid-dump
apt-extracttemplates deb-systemd-helper deb-systemd-invoke debconf          soelim         usbreset
apt-ftpparchive  debconf       glib-compile-schemas lzless         sort           sotruss       users
apt-get          debconf-apt-progress gmake           lzma           split          splitfont      utmpdump
apt-key          debconf-apt-progress gmake           lzma           split          splitfont      uidgen
apt-mark         debconf-apt-progress gmake           lzma           split          splitfont      uidparse
apt-sortpkgs     debconf-apt-progress gmake           lzma           split          splitfont      vdir
```

Lifelines

- ✓ You can write **man** before most default command names to get a comprehensive manual for the command
- ✓ Alternatively, you can use the option **-h** to get a quick guide to the correct usage of that command
- ✓ Pressing Ctrl/CMD+C terminates whatever process you're in
- ✓ You can cycle through your previous commands by pressing the up and down arrows
- ✓ You can use the tab key to autocomplete commands and filenames
- ✓ **Be VERY careful when asking ChatGPT for help!!!**



Summary and next steps

- **Check these resources for extra info and exercises:**
 - **Command cheat sheet:** <https://devhints.io/bash>
 - **Terminal tutorial:** <https://ubuntu.com/tutorials/command-line-for-beginners#3-opening-a-terminal>
 - **Ubuntu for everyone:** <https://cocalc.com/projects/a3734386-82ac-45f1-81b5-dff7f696cecb/files/welcome%20to%20CoCalc.term>
- **Practice with simple exercises!**
 - Create directories, move them...
 - Create simple python scripts, run them, redirect the output...
- **Note: be careful when copying code from forums or ChatGPT!** If not, you might seriously mess up your system. Always double check code before executing!
- **BE VERY CAREFUL WITH rm!!!**

```
jensvn@JenslaptopHP:~$ cowsay Now get practicing!
```

```
-----  
< Now get practicing! >  
-----
```

```
      ^  ^  
      -- --  
     (oo)\  
     ( _ )\      )\\  
           ||-----w||  
           ||         ||
```